

TP Chapitre 3 : Structures et Unions – Pistes

Programmation C Avancée - L3 GLSI

Dr. El Hadji Bassirou TOURÉ
Département de Mathématiques et Informatique
Faculté des Sciences et Techniques
Université Cheikh Anta Diop de Dakar

Note

Ce document contient des pistes et indices pour les exercices du TP. Il ne fournit pas les solutions complètes. L'objectif est de vous guider sans résoudre les problèmes à votre place.

1 Exercice 1 : Manipulation de Structures

Piste 1.1 - create_employee

- Utiliser `malloc(sizeof(Employee))` pour allouer
- Pour email: utiliser `strdup()` ou `malloc + strcpy`
- Vérifier chaque allocation avant de continuer
- En cas d'échec, libérer ce qui a déjà été alloué

```
1 Employee *create_employee(...) {
2     Employee *e = malloc(sizeof(Employee));
3     if (!e) return NULL;
4
5     // Copier les données...
6     e->email = strdup(email);
7     if (!e->email) {
8         free(e);
9         return NULL;
10    }
11
12    return e;
13 }
```

Piste 1.2 - memcmp sur structures

`memcmp` compare octet par octet, incluant les octets de padding. Ces octets peuvent contenir des valeurs arbitraires (non initialisées), même si les membres sont identiques.

Solution: toujours comparer membre par membre, ou initialiser la structure à zéro avec `memset` avant de remplir les champs.

2 Exercice 2 : Analyse Mémoire avec GDB

Piste 2.1 - Commandes GDB utiles

Afficher le type avec les offsets
(gdb) ptype /o struct Data

Examiner N octets en hexadécimal
(gdb) x/16xb &variable

Afficher l'adresse d'un membre
(gdb) print &variable.membre

```
# Calculer la différence d'adresses
(gdb) print (char*)&d.value - (char*)&d.tag
```

Piste 2.2 - Identifier le padding

Dans le dump hexadécimal:

- Les données ont des valeurs connues
- Le padding contient souvent 00 ou des valeurs aléatoires
- Utiliser `offsetof()` pour confirmer les positions

Exemple de layout:

```
struct Data:
  Offset 0: char tag (1 octet)
  Offset 1-3: PADDING (3 octets)
  Offset 4-7: int value (4 octets)
  ...
```

3 Exercice 3 : Unions et Type Punning

Piste 3.1 - IEEE 754

Format IEEE 754 single precision (32 bits):

- Bit 31: signe (0 = positif)
- Bits 30-23: exposant biaisé (+127)
- Bits 22-0: mantisse (partie fractionnaire)

Pour $1.0f = 1.0 \times 2^0$:

- Signe = 0
- Exposant = $0 + 127 = 127 = 0x7F$
- Mantisse = 0 (implicite 1.0)
- Résultat: 0x3F800000

Piste 3.2 - Tagged Union

```
1 Variant var_string(const char *s) {
2     Variant v;
3     v.type = VAR_STRING;
4     v.value.s = strdup(s); // IMPORTANT: copie!
5     return v;
6 }
7
8 void var_free(Variant *v) {
9     if (v->type == VAR_STRING) {
10         free(v->value.s);
11         v->value.s = NULL;
12     }
13 }
```

4 Exercice 4 : Optimisation de Structures

Piste 4.1 - Règle d'optimisation

Trier les membres par taille décroissante:

1. double, long, pointeurs (8 octets)
2. int, float (4 octets)
3. short (2 octets)
4. char (1 octet)

Regrouper les types de même taille pour minimiser le padding interne.

Piste 4.2 - Vérification

```
1 #include <stddef.h>
2
3 printf("sizeof: %zu\n", sizeof(struct X));
4 printf("offset a: %zu\n", offsetof(struct X, a));
5 printf("offset b: %zu\n", offsetof(struct X, b));
6 // ...
```

5 Exercice 5 : Structures Dynamiques

Piste 5.1 - Liste chaînée

Pour list_push_front:

```
1 void list_push_front(List *list, int data) {
2     Node *new_node = malloc(sizeof(Node));
3     new_node->data = data;
4     new_node->next = list->head;
5     list->head = new_node;
6     list->size++;
7 }
```

Pour list_destroy: parcourir et libérer chaque nœud.

Piste 5.2 - Flexible Array Member

```
1 String *string_create(const char *init) {
2     size_t len = strlen(init);
3     // Allouer: sizeof(String) + len + 1
4     String *s = malloc(sizeof(String) + len + 1);
5     s->length = len;
6     s->capacity = len + 1;
7     strcpy(s->data, init);
8     return s;
9 }
```

6 Projet : Simple Database

Piste - Structure Database

```
1 Database *db_create(size_t initial_capacity) {
2     Database *db = malloc(sizeof(Database));
3     if (!db) return NULL;
4
5     db->records = malloc(initial_capacity * sizeof(Record));
6     if (!db->records) {
7         free(db);
8         return NULL;
9     }
10
11     db->count = 0;
12     db->capacity = initial_capacity;
13     db->next_id = 1;
```

```

4
5     return db;
6 }

```

Piste - Redimensionnement

Quand `count == capacity`:

```

1 static int db_grow(Database *db) {
2     size_t new_capacity = db->capacity * 2;
3     Record *new_records = realloc(db->records,
4                                   new_capacity * sizeof(Record));
5     if (!new_records) return -1;
6
7     db->records = new_records;
8     db->capacity = new_capacity;
9     return 0;
10 }

```

Piste - Insertion

```

1 int db_insert(Database *db, const char *name, ...) {
2     if (db->count >= db->capacity) {
3         if (db_grow(db) < 0) return -1;
4     }
5
6     Record *r = &db->records[db->count];
7     r->id = db->next_id++;
8     strncpy(r->name, name, MAX_NAME - 1);
9     r->name[MAX_NAME - 1] = '\0';
10    // ... autres champs
11
12    db->count++;
13    return r->id;
14 }

```

Piste - Recherche

```

1 Record *db_find_by_id(Database *db, int id) {
2     for (size_t i = 0; i < db->count; i++) {
3         if (db->records[i].id == id) {
4             return &db->records[i];
5         }
6     }
7     return NULL;
8 }

```

Piste - Suppression

Deux stratégies possibles:

1. **Compactage**: déplacer le dernier élément à la place du supprimé
2. **Marquage**: marquer comme supprimé (e.g., `id = -1`)

Stratégie compactage:

```

1 int db_delete(Database *db, int id) {
2     for (size_t i = 0; i < db->count; i++) {
3         if (db->records[i].id == id) {
4             // Remplacer par le dernier element
5             if (i < db->count - 1) {
6                 db->records[i] = db->records[db->count - 1];
7             }
8             db->count--;
9             return 0;
10        }
11    }
12    return -1; // non trouve
13 }

```

Piste - Statistiques

```
1 double db_average_salary(const Database *db) {  
2     if (db->count == 0) return 0.0;  
3  
4     double sum = 0.0;  
5     for (size_t i = 0; i < db->count; i++) {  
6         sum += db->records[i].salary;  
7     }  
8     return sum / db->count;  
9 }
```

Piste - Organisation du code

Structure de fichiers recommandée:

```
database.h -- declarations (typedef, prototypes)  
database.c -- implementations  
main.c     -- programme de test  
Makefile   -- compilation
```

Makefile minimal:

```
CC = gcc  
CFLAGS = -Wall -Wextra -Werror -std=c11 -g
```

```
all: database
```

```
database: main.o database.o  
    $(CC) $(CFLAGS) -o $@ $^
```

```
%.o: %.c database.h  
    $(CC) $(CFLAGS) -c $<
```

```
clean:  
    rm -f *.o database
```

```
valgrind: database  
    valgrind --leak-check=full ./database
```